Inference Engineering, Part 2: Sampling From Scratch

In Part 1, we built manual autoregressive inference: a loop that feeds tokens into a model and picks the next one with `argmax`. It works, but it hides something important. A language model never outputs "a token." It outputs a score for *every* token in its vocabulary, and what we do with those scores is a separate decision.

That decision is called **decoding**, and in this part we build the whole sampling pipeline by hand, with no `generate()` call, using `Qwen2.5-0.5B-Instruct` in PyTorch.

The pipeline we will end up with:

```
model forward pass
      ↓
next-token logits
      ↓
temperature scaling
      ↓
top-k filtering
      ↓
softmax
      ↓
top-p filtering + renormalize
      ↓
multinomial sample
```

1. What the model actually gives us

We use the prompt `"The fastest animal in the world is"`, which tokenizes to 7 tokens. One forward pass returns logits of shape:

```
[batch, sequence_length, vocab_size] = [1, 7, 151936]
```

For every position, the model scores all 151,936 tokens in its vocabulary. Only the **last position** predicts the next token, so we slice it out:

```
last_token_logits = outputs.logits[:, -1, :]   # [1, 151936]
```

In `[:, -1, :]`, the first `:` keeps the whole batch, `-1` picks the final position, and the last `:` keeps every vocabulary score.

Inspecting the top 10 with `torch.topk` gives raw logits like `' the'` at 15.477 and `' a'` at 14.617. These are *not* percentages, only relative scores.

2. Logits to probabilities: softmax

Softmax turns the scores into a probability distribution:

```
probs = torch.softmax(last_token_logits, dim=-1)
```

`dim=-1` means "normalize across the vocabulary." After softmax, every value is between 0 and 1, the ranking is unchanged, and everything sums to about 1.

Now the picture is readable: `' the'` has 43.24%, `' a'` has 18.31%, `' likely'` has 4.02%, and so on.

A small gotcha: `probs.sum()` printed `0.9995`, not `1.0`. That is not a bug in softmax. The model is loaded in `float16`, which has limited precision, so summing 151,936 tiny values accumulates rounding error. If you need clean numbers, cast logits to `float32` before softmax.

3. Sampling instead of argmax

Greedy decoding ignores everything except the maximum. Sampling draws a token *according to* the distribution:

```
next_token = torch.multinomial(probs, num_samples=1)   # shape [1, 1]
```

In one run it returned token 264 (`' a'`) even though `' the'` was more likely. That is the point: `' a'` had an 18% chance, so it gets picked about 18% of the time. The same prompt can now produce many different valid continuations.

Greedy: *probabilities → highest token.* Sampling: *probabilities → weighted random token.*

4. Temperature: how much do we trust the ranking?

Temperature divides the logits **before** softmax:

```
scaled_logits = last_token_logits / temperature
probs = torch.softmax(scaled_logits, dim=-1)
```

It never changes the ranking. It changes the *gaps* between logits, and because softmax uses exponentials, gaps matter a lot. Here is what we measured for `' the'`:

Temperature	P(`' the'`)	Effect
0.5	82.81%	Sharper, more predictable
1.0	43.24%	The model's original distribution
1.5	10.75%	Flatter, more random

A quick intuition with toy numbers: logits `10, 8, 6` at T = 0.5 become `20, 16, 12`, so the leader pulls away. At T = 2 they become `5, 4, 3`, so the candidates move closer together.

- **T < 1:** more consistent, good for extraction, structured output, and coding-style tasks.
- **T > 1:** more diverse, useful for brainstorming and creative writing.
- **T → 0:** approaches greedy decoding (serving APIs typically special-case T = 0 as greedy, since you cannot divide by zero).

Temperature reshapes the whole distribution, but it never makes a token *impossible*. For that we need filtering.

5. Top-k: a hard cap on candidates

Top-k keeps only the `k` highest logits and sets everything else to `-inf`:

```
k = 5
top_k_values, _ = torch.topk(last_token_logits, k=k)
threshold = top_k_values[:, -1:]            # the 5th-highest logit, shape
[1, 1]
filtered_logits = last_token_logits.clone()
filtered_logits[filtered_logits < threshold] = float("-inf")
top_k_probs = torch.softmax(filtered_logits, dim=-1)
```

A few details worth understanding:

- `[:, -1:]` (not `[:, -1]`) keeps the dimension, so the threshold is `[1, 1]` and broadcasts cleanly against `[1, 151936]`.
- `.clone()` matters because we are about to modify the tensor in place, and we do not want to destroy the model's original output.
- `tensor[mask] = value` replaces every position where the mask is `True`.
- `exp(-inf) = 0`, so filtered tokens get exactly zero probability, and softmax renormalizes the survivors. `' the'` jumped from 43.24% to 60.84% when only 5 tokens remained.

The limitation: a fixed `k` ignores how confident the model is. If one token has 80%, keeping 50 candidates is wasteful. If the distribution is flat, keeping 5 throws away reasonable options. That is what top-p fixes.

6. Top-p (nucleus) sampling: a dynamic cap

Top-p keeps the **smallest set of tokens whose cumulative probability reaches p**. With `p = 0.9`, a confident step might keep 3 tokens, while an uncertain step might keep dozens.

The implementation, step by step:

```
p = 0.9

sorted_probs, sorted_indices = torch.sort(probs, descending=True)
cumulative_probs = torch.cumsum(sorted_probs, dim=-1)

remove_mask = cumulative_probs > p
remove_mask[:, 1:] = remove_mask[:, :-1].clone()
remove_mask[:, 0] = False

sorted_probs[remove_mask] = 0.0
sorted_probs = sorted_probs / sorted_probs.sum(dim=-1, keepdim=True)
```

Sort. Probabilities are stored by token ID, but top-p needs them from most to least likely. `torch.sort` returns the sorted values *and* `sorted_indices`, which remembers which token ID each sorted position came from.

Cumulative sum. Sorted `0.43, 0.18, 0.10, 0.07, 0.05` becomes `0.43, 0.61, 0.71, 0.78, 0.83`. Each entry says how much probability we have collected if we keep everything up to that point.

The mask and the shift. `cumulative_probs > p` marks tokens past the boundary. But the token that *crosses* 0.9 is the one that gets us to 90%, so removing it would leave us short. Shifting the mask one position right (`remove_mask[:, 1:] = remove_mask[:, :-1].clone()`) keeps that crossing token. The `.clone()` is needed because source and destination overlap, and we must not read values we have already overwritten.

Always keep one. `remove_mask[:, 0] = False` guarantees the top token survives, even if it alone exceeds `p`.

Renormalize. After zeroing the tail, the survivors sum to roughly 0.9. Dividing by their sum brings them back to 1. `keepdim=True` keeps the sum as `[1, 1]` so it broadcasts over `[1, 151936]`. This is why `' the'` rose from about 43% to about 48%.

One more indexing trap. After sorting, positions are no longer token IDs. To decode, we must map back through `sorted_indices[0, position]`. Earlier, running `topk` on a vocabulary-aligned tensor gave us token IDs directly, but after sorting that shortcut no longer holds.

7. Putting it together

Here is the complete sampler: temperature, then top-k, then softmax, then top-p, then renormalize, then multinomial.

```
def sample_next_token(last_token_logits, temperature=1.0, top_k=50,
top_p=0.9):
    # 1. temperature
    logits = last_token_logits / temperature

    # 2. top-k filtering
    top_k_values, _ = torch.topk(logits, k=top_k)
    threshold = top_k_values[:, -1:]
    logits = logits.clone()
    logits[logits < threshold] = float("-inf")

    # 3. logits -> probabilities
    probs = torch.softmax(logits, dim=-1)

    # 4. top-p filtering
    sorted_probs, sorted_indices = torch.sort(probs, descending=True)
    cumulative_probs = torch.cumsum(sorted_probs, dim=-1)
    remove_mask = cumulative_probs > top_p
    remove_mask[:, 1:] = remove_mask[:, :-1].clone()
```

```
    remove_mask[:, 0] = False
    sorted_probs[remove_mask] = 0.0

    # 5. renormalize
    sorted_probs = sorted_probs / sorted_probs.sum(dim=-1, keepdim=True)

    # 6. sample a position in sorted space
    sampled_position = torch.multinomial(sorted_probs, num_samples=1)

    # 7. map back to the real token ID
    return torch.gather(sorted_indices, dim=-1, index=sampled_position)
```

The last step uses `torch.gather` to convert the sampled *sorted position* back into an *original vocabulary ID*.

Running it 10 times on the same logits gave `' the'` six times, with `' likely'`, `' __'`, and `' a'` appearing occasionally. Same model state, different valid continuations, which is exactly what we wanted.

8. Inside the decode loop

Now we replace `argmax` in the Part 1 loop:

```
generated_ids = input_ids.clone()

for _ in range(30):
    with torch.no_grad():
        outputs = model(generated_ids)

    last_token_logits = outputs.logits[:, -1, :]
    next_token = sample_next_token(last_token_logits, temperature=0.8,
top_k=50, top_p=0.9)

    generated_ids = torch.cat([generated_ids, next_token], dim=-1)

    if next_token.item() == tokenizer.eos_token_id:
        break

print(tokenizer.decode(generated_ids[0]))
```

Each iteration appends one token (`[1, 7] → [1, 8] → [1, 9] …`) until we hit EOS or 30 tokens. One run produced a fluent but **factually wrong** continuation about fish swimming at 100 km/h. That is worth remembering: sampling controls *which* token is picked from the model's distribution. It does not make the model correct.

Also note that this loop is intentionally naive. It re-feeds the *entire* sequence on every step, so earlier tokens are recomputed again and again. Fixing that is what the KV cache is for.

9. Why we need all of these controls

There are two bad extremes:

- **Always argmax:** deterministic, but often bland and repetitive in open-ended generation.
- **Sample from the full vocabulary:** diverse, but the long tail of 150k tokens contains unreliable choices that occasionally get picked.

The controls sit between them, and each one answers a different question:

Control	What it does	Question it answers
Temperature	Reshapes probabilities	How strongly do we trust the ranking?
Top-k	Limits candidate count	How many tokens may compete?
Top-p	Limits probability mass	How much of the distribution may participate?
Multinomial	Performs the random draw	Which token do we actually pick?

Temperature alone cannot forbid a token, since a low temperature only makes unlikely tokens *less* likely. Top-k and top-p can set the probability to exactly zero. And top-p adapts to the model's confidence, which a fixed `k` cannot.

Key takeaways

1. Logits are raw scores. Softmax makes them probabilities, and it preserves ranking.
2. `torch.multinomial` is the actual random draw.
3. Temperature divides logits before softmax, and it changes shape rather than ranking.
4. Top-k is a fixed-size cutoff. Top-p is an adaptive, probability-mass cutoff.

5. After sorting for top-p, positions are not token IDs, so map back with `sorted_indices` or `gather`.
6. Use `.clone()` when masking in place, and `keepdim=True` when you need broadcasting.
7. FP16 introduces small numerical drift, so use FP32 for sensitive probability math.

What's next

Our loop recomputes the whole sequence on every step, which is wasteful. In the next part, we will tackle the **KV cache**, so each decode step only processes the newest token.

This is Part 2 of my Inference Engineering series, where I build LLM inference pieces by hand to understand what production engines actually do.